

Embedded Challenge - EN

Technical Challenge

Objective

Evaluate the ability to design architecture, orchestrate containerized services, enable communication between containers, and deliver a solid foundation (Dockerfiles, docker-compose) with clear documentation.

Deliverable 1 – Architecture

Produce a simple diagram that represents:

Container A (C/C++): simulates a QR code reading module via a serial port.

It must receive commands:

- `init` : configure the QR code, serial baud rate, etc.
- `ping` : periodic check to verify the QR code is alive, must respond with `pong`
- `start` : waits for a QR code read until timeout X
- `stop` : cancels if it is waiting

It must respond with a JSON object. The response must be designed so that it can be accessed by Container B (local IPC, REST/gRPC, message brokers, etc.).

Container B (Python): gateway service that:

- Connects to a remote encrypted MQTT broker using certificates (`test.mosquitto.org`)
- Subscribes to topics (e.g., `from_cloud/commands`)
- Publishes MQTT events to the cloud (e.g., QR readings, `from_device/events`)
- Converts MQTT messages into QR code commands and sends them to Container A

The diagram must show local flows, components, and main ports/topics.

Deliverable 2 – Implementation

Container A – QR Serial (C/C++)

C/C++ program that connects to a physical/virtual serial port, responsible for generating and reading text strings from that serial port.

It must:

- Open the serial port specified via environment variables (e.g., `SERIAL_PORT=/dev/ttyS1`)
- Interpret “commands/messages” sent by Container B:

Commands:

- `INIT`
- `PING`
- `START`
- `STOP`

Respond accordingly:

- `START` → waits for a reading until timeout after X time

Example responses:

```
1 | {"qr-data":{"code":"ABC123","ts":1730780000}}
```

OR

```
1 | {"qr-data":{"code":"TIMEOUT","ts":1730780000}}
```

- `STOP` → stops readings and responds with `OK`
- `PING` → responds with `PONG`
- `INIT` → responds with `OK` if properly configured

Additional requirements:

- Allow configuration of parameters (e.g., device path, baud rate) via environment variables
- Log to stdout and/or volumes with timestamps
- Expose communication so Container B can use it
- Handle reopening the serial port in case of error

You may simulate the QR code serial port using `socat` :

```
1 | socat -d -d pty,raw,echo=0 pty,raw,echo=0
```

This generates two connected virtual ports (loopback-style), and data can be mocked or sent to one of them.

Container B – Gateway (Python)

Python service that:

- Connects to the remote MQTT broker
- Subscribes to command topics from the cloud (`from_cloud/command`) and routes these commands to Container A
- Subscribes to events from Container A (`from_device/events`)
- Publishes events to the cloud (`from_device/events`)

It must:

- Handle MQTT reconnections in case of errors
- Publish errors to `from_device/events`

Deliverable 3 — Containerization & Orchestration

- Dockerfiles (one for C/C++ and one for Python)

The `docker-compose.yml` must:

- Simulate the QR code as a serial port (`fake-serial`)
- Start a compiled C application
- Configure dependencies and health checks
- Include volumes for log storage

Example pseudo docker-compose:

```
1 services:
2   fake-serial:
3     image: alpine/socat
4     command: socat -d -d pty,raw,echo=0,link=/tmp/ttyS1
5     pty,raw,echo=0,link=/tmp/ttyS2
6     volumes:
7       - /tmp:/tmp
8
9   qr-c:
10    image: alpine/###
11    command: ./qr-c
12    environment:
13      SERIAL_PORT: /tmp/ttyS1
14    volumes:
15      - /tmp:/tmp
16    depends_on:
17      - fake-serial
18
19   gateway-py:
20    image: python:slim-####
21    command: python app.py
22    environment:
23      MQTT_HOST: test.mosquitto.org
24      MQTT_PORT: 8883
```

```
24 MQTT_CERT: /home/mosquitto.org.crt
25 MQTT_TOPIC_CMD: from_cloud/command
26 MQTT_TOPIC_EVENT: from_device/events
27 volumes:
28   - /tmp:/tmp
29 depends_on:
30   - qr
```

Deliverable 4 — Documentation

README.md with:

- How to run and replicate the setup
 - How to test (hardware dependencies can be mocked)
 - Examples
 - Design decisions and rationale
 - Future improvements
-

Suggested Repository Structure

```
1 .
2 |--- diagrams/
3 |   |--- arquitetura.mmd
4 |--- qr-c/
5 |   |--- src/
6 |   |   |--- main.c
7 |   |--- Dockerfile
8 |--- gateway-py/
9 |   |--- app.py
10 |   |--- requirements.txt
11 |   |--- Dockerfile
12 |--- docker-compose.yml
13 |--- README.md
```